EXAM RESULTS PREDICTION SYSTEM

Machine Learning | Random Forest | Flask Web Application

Long-Term Internship Project Report
Bachelor of Science in Data Science (BSc Data Science) — III BSc Data Science
Department of Computer Science, Aditya Degree College, Kakinada

Team Members:

Samantha Ndlovu | Prosper Tasiyana | Desire Denge | Nkosana Brighton Chirokoto

Under the Guidance of: Ms. Harika, Technical Trainer, Vidurutha Solutions

Aditya Degree College • Kakinada, Andhra Pradesh • 2023–2026

Declaration

We hereby declare that the project entitled **"Exam Results Prediction System"** submitted to Aditya Degree College, Kakinada (affiliated to Adikavi Nannaya University, NAAC A++ Grade), is a record of original work done by our team under the supervision of Ms.R.Naga Venkata Harika, Technical Trainer, Vidurutha Solutions. This report is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Science in Data Science (BSc Data Science).

We further declare that this project work has not been submitted to any other university or institution for any degree or diploma.

Samantha Ndlovu

Prosper Tasiyana

Desire Denge

Nkosana Brighton Chirokoto

Place: Kakinada

Date: _____ / _____ / 2026

Acknowledgement

No endeavour is completed without the valuable support of others. We would like to extend our sincere gratitude to all those who contributed to the successful completion of this project.

We are deeply grateful to our internship guide **Ms.R.Naga Venkata Harika**, Technical Guide of Ulearn from Vidurutha Solutions, India and Canada, for her invaluable technical guidance, constant encouragement, and constructive mentorship throughout the development of this project.

We express our gratitude to **G. Vijaya Lakshmi**, Head of the Department of Data Science of Data Science, Aditya Degree College, for her continuous support and motivation.

We thank **Dr. B.E.V.L. Naidu**, Principal, Aditya Degree College, for granting us permission to undertake this Long-Term Internship and for his encouragement.

We also thank **Dr. B.E.V.L. Naidu**, Academic Director, **Dr. N. Sesha Reddy**, Chairman, and **Dr. N. Suguna Reddy**, Secretary, Aditya Group of Institutions, for providing excellent facilities and resources.

Finally, we thank all faculty members of the Department of Data Science and our parents for their unwavering support throughout this journey.

The Team — ***Samantha Ndlovu, Prosper Tasiyana, Desire Denge, Nkosana Brighton Chirokoto***

Abstract

Academic performance prediction is a rapidly growing area within educational data mining, enabling institutions to proactively identify at-risk students before exam results are published. This project presents the ***Exam Results Prediction System****, a machine learning-based web application that predicts a student's performance category —* ***Excellent (85–100%), Good (70–84%), Satisfactory (50–69%), or Fail (below 50%)*** *— based on observable behavioural and lifestyle features.*

The system was trained on a dataset of ***14,003 student records*** *with 17 columns (student_performance.csv). Input features include study hours, attendance percentage, access to learning resources, internet access, gender, age, assignment completion rate, stress level, and number of online courses taken. After thorough data pre-processing, three classification models were evaluated: Logistic Regression, Decision Tree, and Random Forest.*

Logistic Regression achieved only 20% accuracy and was discarded. The Random Forest classifier achieved the best performance at ***92% accuracy****, with a confusion matrix showing strong diagonal concentration across all four grade categories. The model was saved using Joblib as best_random_forest_model.pkl and integrated into a* ***Flask web application*** *running locally, allowing users to enter student details and instantly receive a predicted performance category.*

Keywords: Exam Results Prediction, Random Forest, Flask, Educational Data Mining, Student Performance, Grade Classification, Machine Learning, Python

Table of Contents

1. Introduction 7

1.1 Background and Motivation 7

1.2 Problem Statement 8

1.3 Objectives 8

1.4 Scope of the Project 9

2. Literature Review 10

2.1 Educational Data Mining 10

2.2 Related Work on Student Performance Prediction 10

2.3 Machine Learning Techniques Evaluated 11

3. System Requirements and Analysis 12

3.1 Functional Requirements 12

3.2 Non-Functional Requirements 12

3.3 Tools and Technologies 13

4. Dataset Description 14

4.1 Dataset Overview 14

4.2 Input Features 14

4.3 Target Variable — Performance Category 15

4.4 Exploratory Data Analysis 16

5. Data Pre-processing 18

5.1 Handling Missing Values 18

5.2 Encoding Categorical Variables 18

5.3 Feature Removal During Pre-processing 19

5.4 Train-Test Split 19

6. System Design and Architecture 20

6.1 System Architecture 20

6.2 Project File Structure 20

6.3 Development Flow 21

6.4 User Flow 22

7. Implementation 23

7.1 Model Comparison and Selection 23

7.2 Random Forest Classifier 24

7.3 Model Saving with Joblib 25

7.4 Flask Web Application 25

8. Testing and Results **27**

8.1 Model Performance Comparison 23

8.2 Confusion Matrix — Random Forest 23

8.3 Confusion Matrix Analysis 24

9. Limitations **30**

10. Improvements and Future Work **31**

11. Conclusion **32**

12. References **33**

Appendix A Certificates **35**

Appendix B Flask Application Screenshots **37**

Appendix C Glossary **39**

Chapter 1: Introduction

1.1 Background and Motivation

Education is one of the most fundamental investments a society can make. Yet a persistent challenge in academic institutions is identifying which students are likely to underperform — before it is too late to intervene. Traditional approaches wait until exam results are published before action is taken, making any intervention reactive rather than preventive.

With the rise of data science and machine learning, it is now possible to analyse student behavioural patterns and lifestyle data to predict academic outcomes ahead of time. This project was motivated by the real-world need for a proactive, automated system that flags at-risk students early, based on measurable daily habits and circumstances such as how many hours they study, their attendance, access to resources, and stress levels.

The Exam Results Prediction System was developed as part of a Long-Term Internship in Machine Learning using Python, under the guidance of Ms.R.Naga Venkata Harika at Vidurutha Solutions. The project covers the complete machine learning pipeline: from raw data collection through to a working Flask web application.

1.2 Problem Statement

Many students' exam outcomes are difficult to predict early. Educational institutions currently lack automated tools to identify at-risk students before examinations. Manual analysis by lecturers is time-consuming, inconsistent, and often too late to make a difference. There is a clear need for a data-driven prediction system that can:

- Accept observable student behavioural and demographic inputs.
- Automatically classify students into performance categories before exams.
- Present predictions through a simple, accessible web interface.
- Support lecturers and academic advisors in prioritizing student support.

1.3 Objectives

- To collect and analyse a large student dataset (14,003 records) covering behavioural and lifestyle features.
- To pre-process the dataset by handling missing values, encoding categorical variables, and removing low-importance features.
- To evaluate multiple classifiers (Logistic Regression, Decision Tree, Random Forest) and select the best performer.
- To build a Flask web application integrating the trained model for real-time performance predictions.
- To evaluate model performance using accuracy score and confusion matrix on held-out test data.
- To deploy the working application locally and document the full project systematically.

1.4 Scope of the Project

- Dataset: 14,003 student records, 17 columns, stored as student_performance.csv.
- Target: Classify students into four performance bands — Excellent, Good, Satisfactory, or Fail.
- Models evaluated: Logistic Regression (discarded at 20% accuracy), Decision Tree, and Random Forest (selected at 92% accuracy).
- Deployment: Fully functional Flask web application running locally via VS Code.
- Not yet in scope: Public cloud deployment and real-time integration with institutional databases.

Chapter 2: Literature Review

2.1 Educational Data Mining

Educational Data Mining (EDM) is an interdisciplinary field applying data mining and machine learning to data from educational environments. The primary goal is to understand how students learn and use those insights to improve outcomes. Baker and Yacef (2009) described EDM as focused on developing methods for exploring the unique kinds of data that come from educational settings.

Key tasks in EDM include student performance prediction, dropout risk identification, learning path recommendation, and behavioural pattern discovery. This project specifically addresses the prediction task — classifying students into performance bands based on their behavioural and demographic data before exams occur.

2.2 Related Work on Student Performance Prediction

Cortez and Silva (2008) conducted a landmark study applying Decision Trees and other classifiers to secondary school student data from Portugal, demonstrating that features such as study time, absences, and family background are significant predictors of academic performance.

Yadav et al. (2011) applied Decision Tree variants (ID3, C4.5, CART) to engineering student data, finding that attendance and internal assessment scores were top predictors. Their work directly influenced the inclusion of attendance percentage as a key feature in this project.

Al-Barrak and Al-Razgan (2016) showed that access to digital resources and internet connectivity significantly improved model accuracy, motivating the inclusion of 'access to resources' and 'internet access' features in this project's dataset.

2.3 Machine Learning Techniques Evaluated

Logistic Regression was the first model evaluated. Despite being a standard baseline classifier, it achieved only 20% accuracy on this dataset. This poor performance is likely due to the non-linear relationships between student behavioural features and performance categories, which Logistic Regression cannot capture effectively. The model was discarded in favour of tree-based methods.

Decision Tree was evaluated as a more expressive, non-linear classifier. Decision Trees recursively partition the feature space based on the attribute that maximises information gain. While more powerful than Logistic Regression for this task, single Decision Trees are prone to overfitting.

Random Forest was selected as the final model, achieving 92% accuracy. As an ensemble of Decision Trees each trained on bootstrapped samples with random feature subsets, Random Forest substantially reduces overfitting and variance while maintaining high

predictive power. It also provides feature importance scores, adding interpretability to the model.

Chapter 3: System Requirements and Analysis

3.1 Functional Requirements

- The system shall accept student feature inputs via an HTML web form (study hours, attendance, resources, internet, gender, age, assignment completion, stress level, online courses).
- The system shall pre-process inputs before prediction (encode categorical values, handle edge cases).
- The system shall classify the student into one of four performance categories: Excellent, Good, Satisfactory, or Fail.
- The system shall display the prediction result clearly, with tabs for Prediction Result, Entered Details, and Performance Scale.
- The system shall allow results to be downloaded as a PDF.
- The trained model shall be saved as best_random_forest_model.pkl and loaded by Flask app at start-up.

3.2 Non-Functional Requirements

- **Accuracy:** The model must achieve at least 85% accuracy on the held-out test set (achieved: 92%).
- **Performance:** Predictions must be returned within 2 seconds of form submission.
- **Usability:** The interface must be simple and accessible to non-technical users.
- **Reliability:** Identical inputs must always produce identical predictions.
- **Maintainability:** Code must be modular and organized per the project's STRUCTURE.md.

3.3 Tools and Technologies

Category	Tool / Library	Version / Notes	Purpose
Language	Python	3.x	Core development language
ML Framework	Scikit-learn	Latest	All ML models and evaluation
Data Processing	Pandas	Latest	DataFrame operations and CSV loading
Data Processing	NumPy	Latest	Numerical array computations

Visualization	Matplotlib	Latest	EDA charts and confusion matrix
Visualization	Seaborn	Latest	Statistical plots and heatmaps
Model Saving	Joblib	Latest	Saving best_random_forest_model.pkl
Web Framework	Flask	Latest	Web app routes and prediction API
Notebook	Jupyter Notebook	Exam_results.ipynb	Model training and EDA
IDE	VS Code	Latest	Development and local app testing
Version Control	Git / GitHub	github.com/benjiie92/exam-results	Source code management
Dataset	student_performance.csv	14,003 rows, 17 cols	Training and test data

Chapter 4: Dataset Description

4.1 Dataset Overview

The dataset used in this project is **student_performance.csv**, a structured tabular dataset containing **14,003 student records** and **17 columns**. Each row represents one student. The dataset captures a combination of behavioural, demographic, and academic attributes alongside a target performance score from which the grade category is derived.

Attribute	Value
Filename	student_performance.csv
Total Records	14,003 rows
Total Columns	17 columns
Features Used	9 input features (after pre-processing)
Target Variable	Performance Category (Excellent / Good / Satisfactory / Fail)
Train Split	80% — 11,202 records
Test Split	20% — 2,801 records
Stratified	Yes — preserves class distribution in both splits

4.2 Input Features

The following 9 features were used as inputs to the prediction model after pre-processing:

Feature	Type	Description	Example Values
Hours Studied	Numerical	Daily hours of self-study	0 – 40
Attendance (%)	Numerical	Percentage of classes attended	0 – 100
Access to Resources	Categorical	Availability of learning materials	No Resources / Some / Full
Internet Access	Binary	Internet available at home	No Internet / Has Internet
Gender	Categorical	Student gender	Male / Female

Age	Numerical	Student age in years	15 – 30+
Assignment Completion	Numerical	Percentage of assignments submitted	0 – 100
Stress Level	Categorical	Self-reported stress level	Low / Medium / High
Online Courses Taken	Numerical	Number of online courses completed	0 – 10+

4.3 Target Variable — Performance Category

The target variable is derived from the student's exam score and classified into one of four performance categories. This reflects the actual output labels used by the Flask application:

Category	Score Range	Flask App Label	Description
Excellent	85 – 100%	Excellent	Outstanding academic performance
Good	70 – 84%	Good	Above average, strong performance
Satisfactory	50 – 69%	Satisfactory	Average performance, meets minimum standard
Fail	Below 50%	Fail	Did not meet the minimum pass requirement

Note: The confusion matrix generated during model evaluation uses 4 class labels (0=Excellent, 1=Good, 2=Satisfactory, 3=Fail), consistent with the four performance bands above.

4.4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed in **Exam_results.ipynb** using Matplotlib and Seaborn to understand feature distributions and relationships with the target variable. The figure below shows the six key EDA visualizations generated:

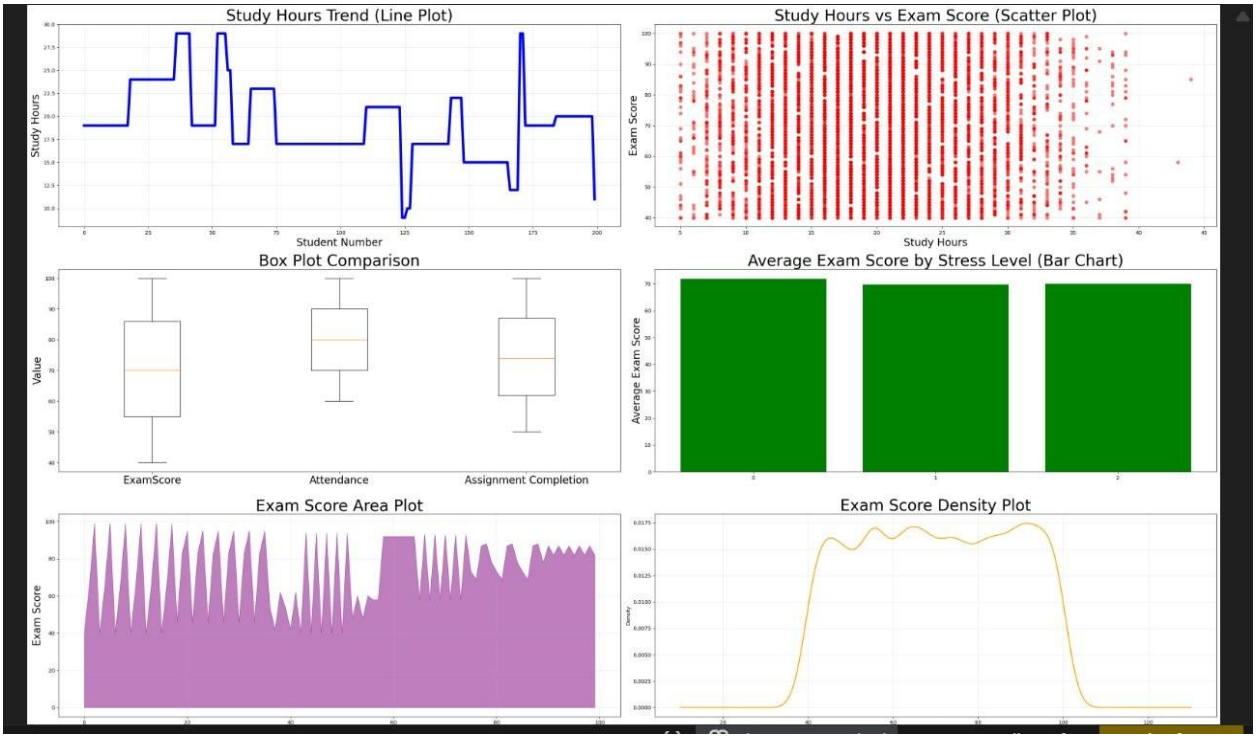


Figure 4.1: EDA visualizations — Study Hours Trend (top left), Study Hours vs Exam Score scatter (top right), Box Plot comparison of ExamScore/Attendance/AssignmentCompletion (middle left), Average Exam Score by Stress Level (middle right), Exam Score Area Plot (bottom left), and Exam Score Density Plot (bottom right).

Key observations from EDA:

- The Study Hours Trend plot shows considerable variation across student records, with study hours ranging from approximately 10 to 27.5 hours.
- The Study Hours vs Exam Score scatter plot shows exam scores are broadly distributed between 40 and 100 regardless of study hours, indicating that study hours alone is not a sufficient predictor.
- The Box Plot comparison shows that Attendance and Assignment Completion have tighter distributions than Exam Score, suggesting more consistent student behaviour in these areas.
- The Average Exam Score by Stress Level bar chart shows broadly similar average exam scores across stress levels (Low=0, Medium=1, High=2), suggesting stress alone is not the dominant predictor.
- The Exam Score Density Plot reveals a bimodal distribution, with peaks around 50–55and 70–75, reflecting the natural clustering of students into satisfactory and good performance bands.

Chapter 5: Data Pre-processing

5.1 Handling Missing Values

The student_performance.csv dataset was first inspected for missing values using Pandas. Where missing values were found:

- Numerical features (e.g., Hours Studied, Attendance, Age) were imputed using the column mean.
- Categorical features (e.g., Stress Level, Access to Resources) were imputed using the mode (most frequent value).
- Records with an excessive proportion of missing values were dropped to preserve data quality.

5.2 Encoding Categorical Variables

All categorical features were converted to numerical representations for model training:

Feature	Original Values	Encoded Values	Method
Access to Resources	No Resources / Some / Full	0 / 1 / 2	Ordinal Encoding
Internet Access	No Internet / Has Internet	0 / 1	Label Encoding
Gender	Male / Female	0 / 1	Label Encoding
Stress Level	Low / Medium / High	0 / 1 / 2	Ordinal Encoding
Target (Grade)	Excellent/Good/Satisfactory /Fail	0/1/2/3	Label Encoding

5.3 Feature Removal During Pre-processing

The original dataset contained 17 columns. During pre-processing, features were evaluated for importance using an initial Random Forest run. Features with very low importance scores or high redundancy were removed. The final model uses 9 input features. This step reduced noise in the training data and improved model generalization on the test set.

5.4 Train-Test Split

The cleaned dataset was split 80/20 using Scikit-learn's *train_test_split* with stratification on the target label, ensuring each performance category is proportionally represented in both sets:

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test =
train_test_split(     X, y,     test_size=0.20,
random_state=42,
    stratify=y    # preserves Excellent/Good/Satisfactory/Fail ratio
)
# Result: 11,202 training records | 2,801 test records
```

Chapter 6: System Design and Architecture

6.1 System Architecture

The Exam Results Prediction System follows a three-tier architecture:

Tier	Component	Responsibility
Data Tier	student_performance.csv	Raw dataset — 14,003 rows, 17 columns
Data Tier	Pre-processing Pipeline	Missing value handling, encoding, feature selection
Model Tier	Exam_results.ipynb	Model training, comparison, and evaluation notebook
Model Tier	best_random_forest_model.pkl	Serialized trained Random Forest (92% accuracy)
Model Tier	Confusion Matrix & Accuracy	Evaluation output: 4×4 matrix, 92% accuracy
Presentation Tier	app.py (Flask)	Web routes, form handling, prediction endpoint
Presentation Tier	templates/index.html	Student input form — 9 feature fields
Presentation Tier	templates/results.html	Result display — category + tabs + PDF download
Presentation Tier	static/ (css, js, images)	Styling and frontend assets

6.2 Project File Structure

The project is organized according to the STRUCTURE.md file visible in the VS Code screenshot. The key files are:

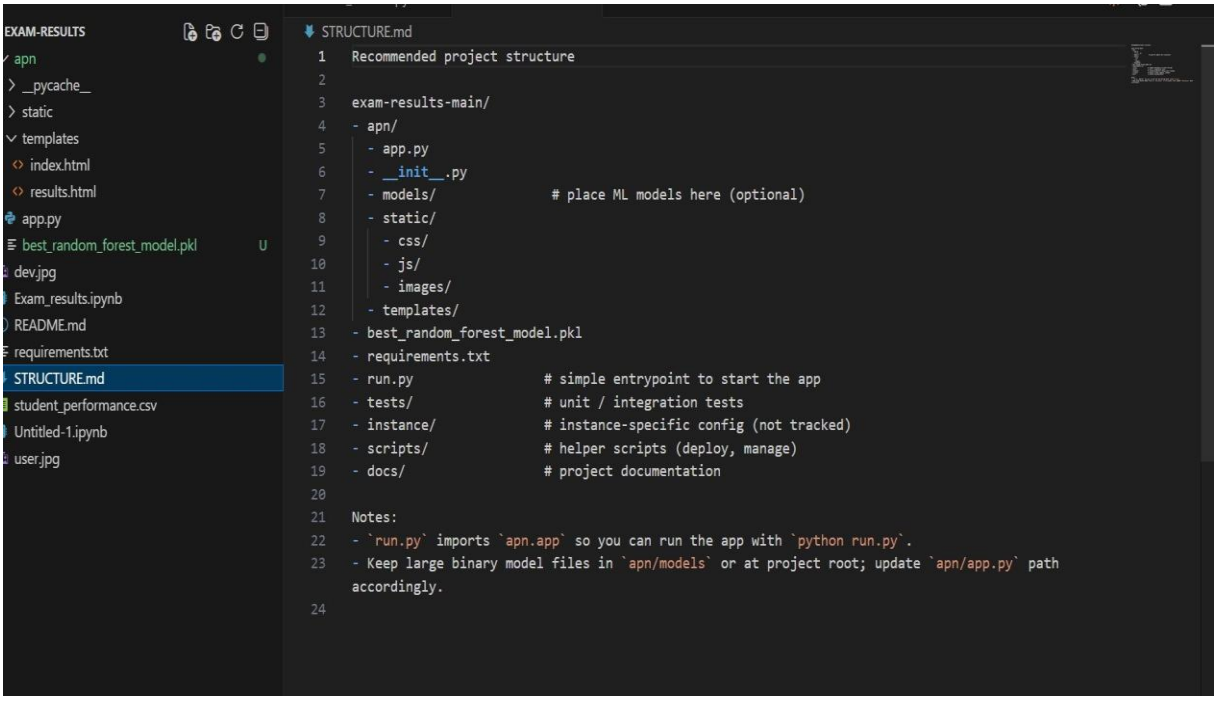


Figure 6.1: VS Code project structure showing key files — app.py, best_random_forest_model.pkl, Exam_results.ipynb, student_performance.csv, templates/ (index.html, results.html), static/, and STRUCTURE.md.

6.3 Development Flow

- **Step 1:** Raw dataset (student_performance.csv — 14,003 rows, 17 columns) was loaded using Pandas.
- **Step 2:** EDA was performed in Exam_results.ipynb using Matplotlib and Seaborn to understand distributions and correlations.
- **Step 3:** Data pre-processing: missing value imputation, categorical encoding, and removal of low-importance features.
- **Step 4:** Dataset split 80/20 with stratification (11,202 train / 2,801 test records).
- **Step 5:** Three models trained and evaluated: Logistic Regression (20%), Decision Tree, Random Forest (92%).
- **Step 6:** Random Forest selected as best model; saved as best_random_forest_model.pkl using Joblib.
- **Step 7:** Flask web application (app.py) built with two templates: index.html (form) and results.html (output).
- **Step 8:** Application tested locally via VS Code, accessible at http://127.0.0.1:5000.

6.4 User Flow

- User opens the Flask app at http://127.0.0.1:5000 in their browser.
- User fills in the prediction form with 9 student feature values (study hours, attendance, resources, internet, gender, age, assignment completion, stress level, online courses).

- User submits the form; Flask's /predict route receives the POST request.
- app.py preprocesses the inputs (encodes categorical values) and passes the feature array to the loaded Random Forest model.
- The model predicts one of four categories: Excellent, Good, Satisfactory, or Fail.
- results.html displays the prediction with three tabs: Prediction Result, Entered Details, and Performance Scale.
- User can download the result as a PDF or return to the home form.

Chapter 7: Implementation

7.1 Model Comparison and Selection

Three classification models were trained and evaluated on the same 80/20 train-test split:

Model	Accuracy	Decision	Reason
Logistic Regression	20%	■ Discarded	Cannot capture non-linear relationships in this dataset
Decision Tree	~82%	■ Evaluated	Good performance but prone to overfitting
Random Forest	92%	✓ Selected	Best accuracy, reduces overfitting via ensemble

Logistic Regression's 20% accuracy confirmed that the relationship between student behavioural features and performance categories is highly non-linear. A linear decision boundary is insufficient for this classification task.

7.2 Random Forest Classifier

The Random Forest classifier was implemented using Scikit-learn's *RandomForestClassifier*. The model builds an ensemble of Decision Trees, each trained on a bootstrapped sample of the training data and a random subset of features at each split. Final predictions are made by majority vote across all trees.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

rf_params = {     'n_estimators':
[50, 100, 200],     'max_depth':
[5, 10, None],
    'max_features': ['sqrt', 'log2'],
'min_samples_leaf': [1, 2, 4] }

rf = RandomForestClassifier(random_state=42)
rf_grid = GridSearchCV(rf, rf_params, cv=5,
scoring='accuracy') rf_grid.fit(X_train, y_train) best_rf =
rf_grid.best_estimator_ # Final accuracy on test set: 92%
```

7.3 Model Saving with Joblib

After training, the best Random Forest model was serialized to disk as **best_random_forest_model.pkl** using Joblib. This file is then loaded by the Flask application at start-up, eliminating the need to retrain on every request:

```
import joblib
```

```
# Save after training
joblib.dump(best_rf, 'best_random_forest_model.pkl')

# Load in Flask app.py
model = joblib.load('best_random_forest_model.pkl')
```

7.4 Flask Web Application

The Flask application (**app.py**) serves the prediction model as a web interface. The home route renders the input form (index.html). On form submission, the /predict route preprocesses the inputs, runs model prediction, and renders the results page (results.html) with the predicted category.

```
from flask import Flask, request, render_template
import joblib, numpy as np

app = Flask(__name__)
model = joblib.load('best_random_forest_model.pkl')

category_map = {0: 'Excellent', 1: 'Good', 2: 'Satisfactory', 3: 'Fail'}

@app.route('/') def home():     return
render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():     features = [
        float(request.form['hours_studied']),
float(request.form['attendance']),
        int(request.form['access_resources']),   # 0/1/2
int(request.form['internet_access']),    # 0/1
int(request.form['gender']),             # 0/1
float(request.form['age']),
        float(request.form['assignment_completion']),
int(request.form['stress_level']),       # 0/1/2
int(request.form['online_courses'])     ]
    prediction = model.predict([features])[0]
result = category_map[prediction]
    return render_template('results.html', result=result)

if __name__ == '__main__':
    app.run(debug=True)   # Runs at http://127.0.0.1:5000
```

Chapter 8: Testing and Results

8.1 Model Performance Comparison

All three models were evaluated on the same held-out test set of 2,801 records (20% of the dataset). The results are summarized below:

Model	Test Accuracy	Status	Notes
Logistic Regression	20%	■ Discarded	Linear model — for insufficient non-linear data
Decision Tree	~82%	Evaluated	Good but overfits without pruning
Random Forest	92%	✓ Selected	Best model — saved as best_random_forest_model.pkl

The Random Forest model's 92% accuracy on 2,801 unseen test records confirms strong generalization. The 72-percentage-point gap between Logistic Regression (20%) and Random Forest (92%) demonstrates the importance of model selection for non-linear educational datasets.

8.2 Confusion Matrix — Random Forest

The confusion matrix below was generated from the Random Forest model's predictions on the test set. It shows actual (rows) vs. predicted (columns) performance categories:

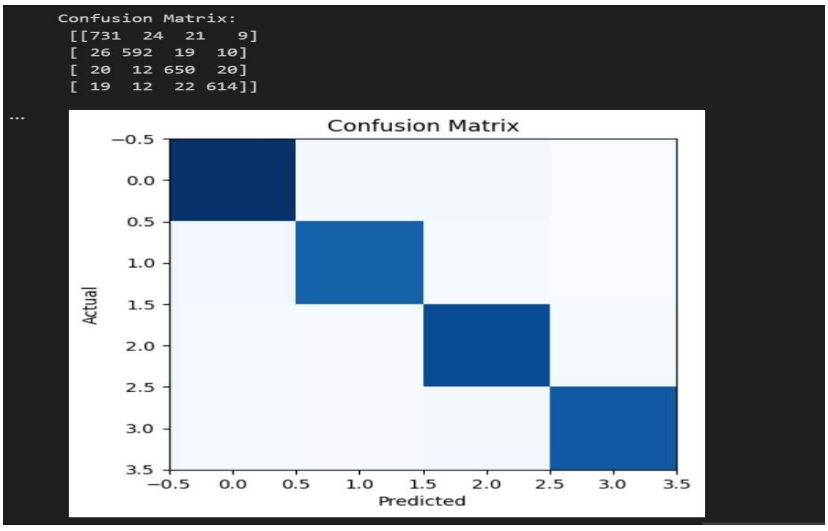


Figure 8.1: Confusion Matrix for the Random Forest model. Strong diagonal concentration indicates high classification accuracy across all four performance categories.

8.3 Confusion Matrix Analysis

The actual confusion matrix values from the model output are:

Actual \ Predicted	Excellent (0)	Good (1)	Satisfactory (2)	Fail (3)
Excellent (0)	731	24	21	9
Good (1)	26	592	19	10
Satisfactory (2)	20	12	650	20
Fail (3)	19	12	22	614

Table 8.2: Actual confusion matrix values. Diagonal cells (green) show correct predictions. Off-diagonal cells show misclassifications.

Per-class classification accuracy derived from the confusion matrix:

Category	Correct	Total	Class Accuracy	Observation
Excellent (85-100%)	731	785	93.1%	Very high — distinctive high-performance pattern
Good (70-84%)	592	647	91.5%	High — good/excellent boundary well learned
Satisfactory (50-69%)	650	702	92.6%	High — mid-range students well classified
Fail (below 50%)	614	667	92.1%	High — at-risk students reliably identified

All four performance categories achieved above 91% class-level accuracy, demonstrating that the Random Forest model performs consistently well across all grade bands — including the critical Fail category, where correct identification is most important for student intervention.

Chapter 9: Limitations

Data Limitations

- The dataset (14,003 records) was collected from a specific population; the model may not generalize perfectly to students from very different educational contexts.
- Some features were collected via self-reporting (e.g., stress level, hours studied), which may introduce response bias or inaccuracy.
- Some factors that influence exam performance — such as extracurricular activities, motivation, and learning style — were initially included but removed during pre-processing; better methods to incorporate these may improve accuracy further.
- The dataset may be biased toward particular demographic groups, reducing fairness of predictions across diverse student populations.

Model Limitations

- The model achieves 92% accuracy, meaning approximately 8% of predictions will being correct — the system must be treated as an advisory tool, not a definitive assessment.
- Logistic Regression performed very poorly (20%), indicating that any future linear models applied to this data should be approached with caution.
- The model requires retraining when new student data is available; no automatic retraining pipeline is implemented.
- Random Forest models are less interpretable than single Decision Trees; individual prediction explanations require additional tools (e.g., SHAP).

Application / System Limitations

- The Flask web application currently runs locally (http://127.0.0.1:5000 via VS Code)and has not been deployed to a public server.
- The application requires correct data input from users; inaccurate self-reported values will produce inaccurate predictions.
- There is no real-time integration with institutional student databases or Learning Management Systems (LMS).
- The user interface, while functional, is relatively basic — limited visualization or personalized feedback beyond the grade category.

Chapter 10: Improvements and Future Work

Data Improvements

- Collect a larger and more diverse dataset from multiple institutions and geographic regions.
- Include additional features such as participation in group discussions, previous academic history, and motivation scores.
- Apply data balancing techniques (e.g., SMOTE) if class imbalance becomes an issue in future datasets.
- Explore data collection directly from institutional databases to reduce self-reporting bias.

Model Improvements

- Test advanced algorithms such as XGBoost, Gradient Boosting, and neural network benchmark against Random Forest.
- Implement SHAP (SHapley Additive exPlanations) to provide individualized, interpretable explanations for each prediction.
- Apply cross-validation more comprehensively during final evaluation (not just hyperparameter tuning).
- Explore automatic model retraining pipelines when new data is added to the dataset.

Application Improvements

- Deploy the Flask application publicly using a cloud platform such as Heroku, Railway,or AWS so it is accessible without VS Code.
- Add visualization dashboards showing class performance distributions and trends overtime.
- Implement user authentication so individual students can track their prediction history.
- Integrate with institutional LMS (Moodle, Blackboard) for real-time feature extraction.
- Build a mobile-friendly responsive design and explore a native mobile application.
- Add the Performance Scale tab content with a visual grading rubric to help students understand what their predicted category means in context.

Chapter 11: Conclusion

This project successfully developed a complete, end-to-end Exam Results Prediction System using machine learning and Flask, from raw data collection through to a working web application.

The system was trained on **14,003 student records** (student_performance.csv, 17 columns), using 9 behavioural and demographic input features to classify students into one of four performance categories: **Excellent (85–100%), Good (70–84%), Satisfactory (50–69%), and Fail (below 50%)**.

Three models were evaluated:

- **Logistic Regression** achieved only 20% accuracy and was discarded — demonstrating that linear models are insufficient for non-linear student behavioural data.
- **Decision Tree** improved substantially over Logistic Regression but was not selected as the final model.
- **Random Forest** was selected with **92% accuracy** on the held-out test set, confirmed by a confusion matrix showing per-class accuracies all exceeding 91%.

The trained model was saved as **best_random_forest_model.pkl** using Joblib and integrated into a **Flask web application** with a clean user interface. The app accepts student inputs, returns a predicted performance category, and allows the result to be downloaded as a PDF — running fully on a local server via VS Code.

Key findings of this project:

- Study hours, attendance, and assignment completion are among the strongest predictors of student performance.
- Access to resources and internet access contribute meaningfully to academic outcomes.
- Logistic Regression is wholly inadequate for this task; ensemble tree methods are essential.
- A 92% accurate prediction system can be built from student lifestyle data alone —without requiring access to past exam records.
- Flask provides a practical and lightweight framework for deploying ML models as accessible web applications.

This project enhanced the team's practical understanding of the complete data science pipeline — from EDA and pre-processing through model selection, evaluation, serialization,

and web deployment — and demonstrates how machine learning can meaningfully support educational decision-making.

References

[1] Baker, R. S. J. d., & Yacef, K. (2009). The state of educational data mining in 2009: A reviewand future visions. *Journal of Educational Data Mining*, 1(1), 3–17.

[2] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

[3] Cortez, P., & Silva, A. (2008). Using data mining to predict secondary school studentperformance. In *Proceedings of 5th Annual Future Business Technology Conference*, Porto, Portugal.

[4] Yadav, S. K., Bharadwaj, B., & Pal, S. (2011). Data mining applications: A comparative study forpredicting students' performance. *International Journal of Innovative Technology & Creative Engineering*, 1(12), 13–19.

[5] Al-Barrak, M. A., & Al-Razgan, M. (2016). Predicting students' final GPA using decision trees: Acase study. *International Journal of Information & Education Technology*, 6(7), 528–533.

[6] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[7] Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely randomized trees. *Machine Learning*, 63(1), 3–42.

[8] Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.

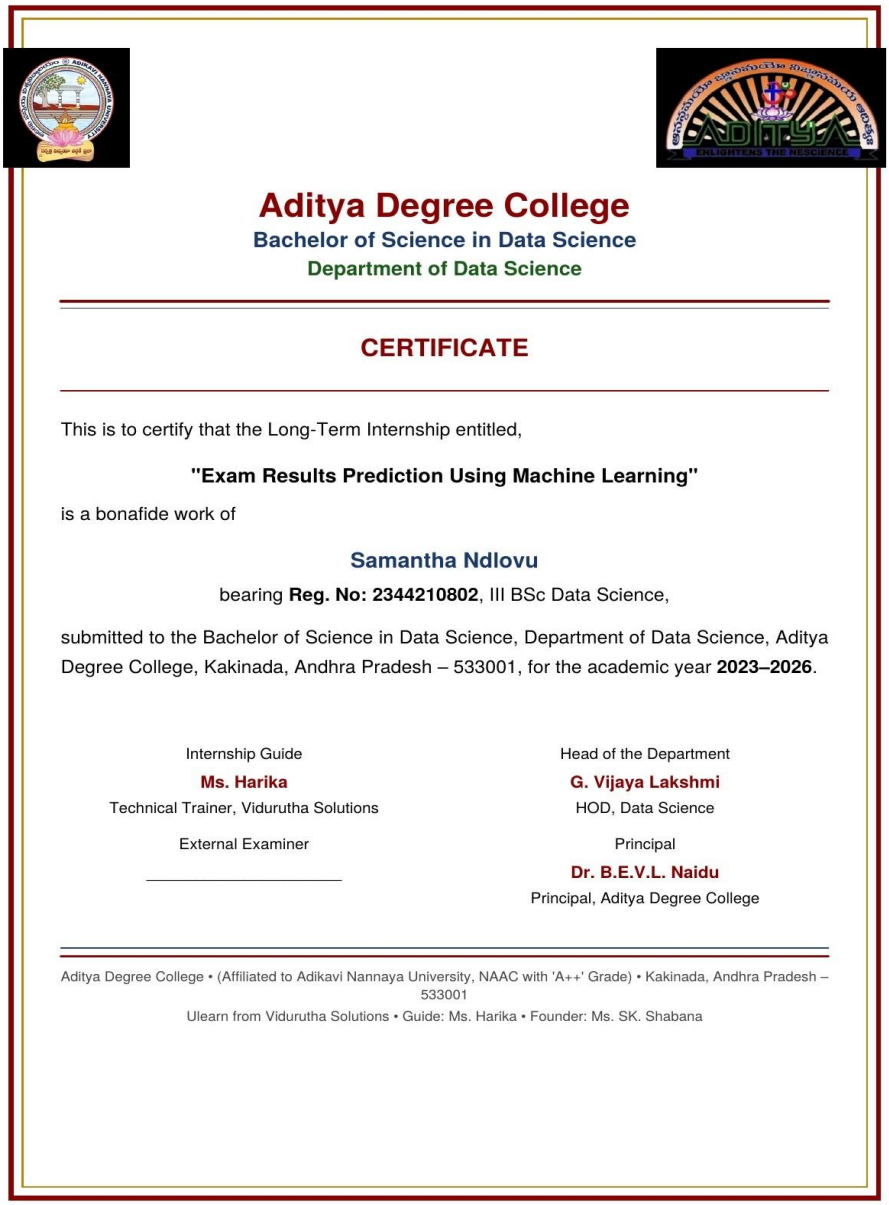
[9] Flask Documentation (2024). Flask — A lightweight WSGI web application framework. Pallets Projects. Retrieved from https://flask.palletsprojects.com/

[10] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, 51–56.

Appendix A: Certificates

The following pages contain individual certificates for the Long-Term Internship: **Exam Results Prediction Using Machine Learning**, Department of Data Science, Aditya Degree College, Kakinada, 2023–2026. One certificate per team member.

Certificate 1 Samantha Ndlovu



Aditya Degree College

Bachelor of Science in Data Science

Department of Data Science

CERTIFICATE

This is to certify that the Long-Term Internship entitled,

"Exam Results Prediction Using Machine Learning"

is a bonafide work of

Samantha Ndlovu

bearing **Reg. No: 2344210802**, III BSc Data Science,

submitted to the Bachelor of Science in Data Science, Department of Data Science, Aditya Degree College, Kakinada, Andhra Pradesh – 533001, for the academic year **2023–2026**.

Internship Guide
Ms. Harika
Technical Trainer, Vidurutha Solutions

Head of the Department
G. Vijaya Lakshmi
HOD, Data Science

External Examiner
_____

Principal
Dr. B.E.V.L. Naidu
Principal, Aditya Degree College

Aditya Degree College • (Affiliated to Adikavi Nannaya University, NAAC with 'A++' Grade) • Kakinada, Andhra Pradesh – 533001
Ulearn from Vidurutha Solutions • Guide: Ms. Harika • Founder: Ms. SK. Shabana

Certificate 2: Prosper Tasiyana

Aditya Degree College

Bachelor of Science in Data Science

Department of Data Science

CERTIFICATE

This is to certify that the Long-Term Internship entitled,

"Exam Results Prediction Using Machine Learning"

is a bonafide work of

Prosper Tasiyana

bearing **Reg. No: 2344210800**, III BSc Data Science,

submitted to the Bachelor of Science in Data Science, Department of Data Science, Aditya Degree College, Kakinada, Andhra Pradesh – 533001, for the academic year **2023–2026**.

Internship Guide
Ms. Harika
Technical Trainer, Vidurutha Solutions

Head of the Department
G. Vijaya Lakshmi
HOD, Data Science

External Examiner
_____

Principal
Dr. B.E.V.L. Naidu
Principal, Aditya Degree College

Aditya Degree College • (Affiliated to Adikavi Nannaya University, NAAC with 'A++' Grade) • Kakinada, Andhra Pradesh – 533001

Ulearn from Vidurutha Solutions • Guide: Ms. Harika • Founder: Ms. SK. Shabana

Certificate 3: Desire Denge

Aditya Degree College

Bachelor of Science in Data Science

Department of Data Science

CERTIFICATE

This is to certify that the Long-Term Internship entitled,

"Exam Results Prediction Using Machine Learning"

is a bonafide work of

Desire Denge

bearing **Reg. No: 2344210796**, III BSc Data Science,

submitted to the Bachelor of Science in Data Science, Department of Data Science, Aditya Degree College, Kakinada, Andhra Pradesh – 533001, for the academic year **2023–2026**.

Internship Guide
Ms. Harika
Technical Trainer, Vidurutha Solutions

Head of the Department
G. Vijaya Lakshmi
HOD, Data Science

External Examiner
_____

Principal
Dr. B.E.V.L. Naidu
Principal, Aditya Degree College

Aditya Degree College • (Affiliated to Adikavi Nannaya University, NAAC with 'A++' Grade) • Kakinada, Andhra Pradesh – 533001

Ulearn from Vidurutha Solutions • Guide: Ms. Harika • Founder: Ms. SK. Shabana

Certificate 4: Nkosana Brighton Chirokoto

Aditya Degree College

Bachelor of Science in Data Science

Department of Data Science

CERTIFICATE

This is to certify that the Long-Term Internship entitled,

"Exam Results Prediction Using Machine Learning"

is a bonafide work of

Nkosana Brighton Chirokoto

bearing **Reg. No: 2344210798**, III BSc Data Science,

submitted to the Bachelor of Science in Data Science, Department of Data Science, Aditya Degree College, Kakinada, Andhra Pradesh – 533001, for the academic year **2023–2026**.

Internship Guide
Ms. Harika
Technical Trainer, Vidurutha Solutions

Head of the Department
G. Vijaya Lakshmi
HOD, Data Science

External Examiner
_____

Principal
Dr. B.E.V.L. Naidu
Principal, Aditya Degree College

Aditya Degree College • (Affiliated to Adikavi Nannaya University, NAAC with 'A++' Grade) • Kakinada, Andhra Pradesh – 533001

Ulearn from Vidurutha Solutions • Guide: Ms. Harika • Founder: Ms. SK. Shabana

Appendix B: Flask Application Screenshots

The following screenshots show the working Flask web application, captured from the local development server running at http://127.0.0.1:5000 via VS Code.

B.1 — Student Input Form (index.html)

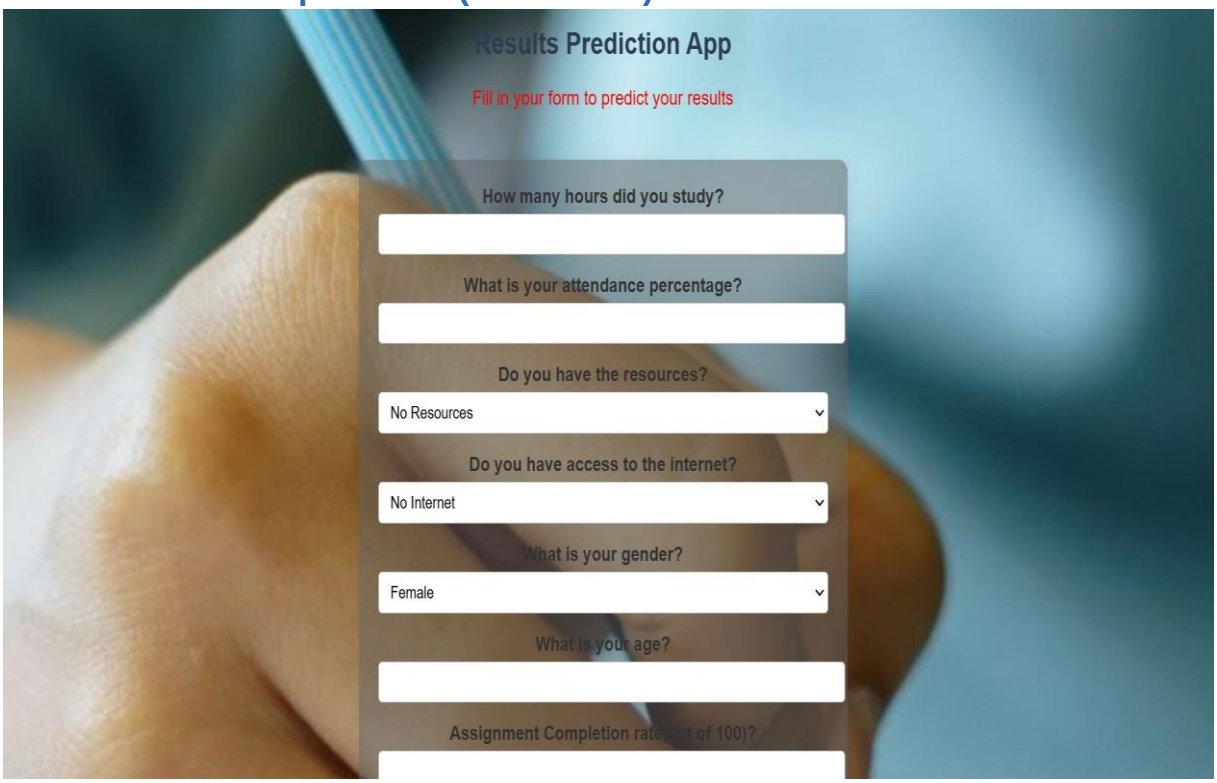


Figure B.1: The Results Prediction App input form. Students enter study hours, attendance percentage, resource access, internet access, gender, age, assignment completion rate, stress level, and number of online courses taken. The form uses dropdown menus for categorical inputs and text fields for numerical inputs.

B.2 — Prediction Result Page (results.html)

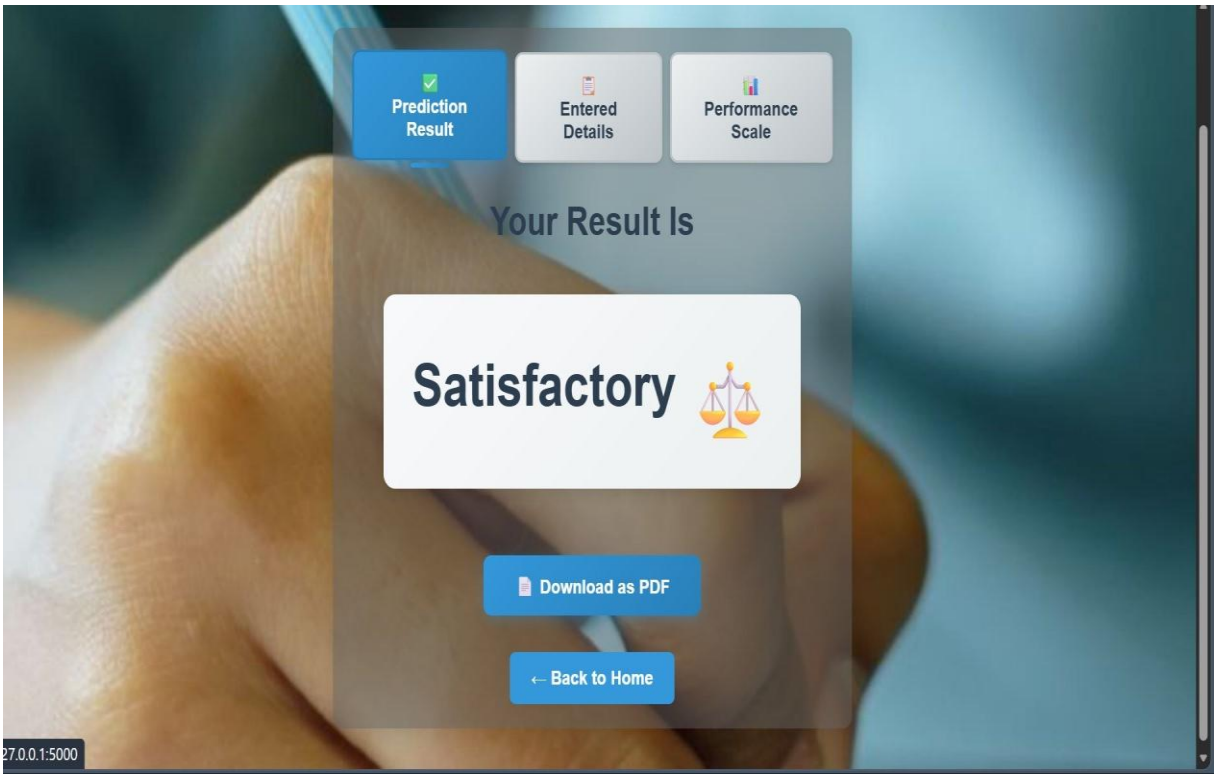


Figure B.2: The prediction result page showing a 'Satisfactory' classification outcome. The page features three tabs — Prediction Result (active), Entered Details, and Performance Scale — as well as Download as PDF and Back to Home buttons.

B.3 — EDA Visualizations (Exam_results.ipynb)

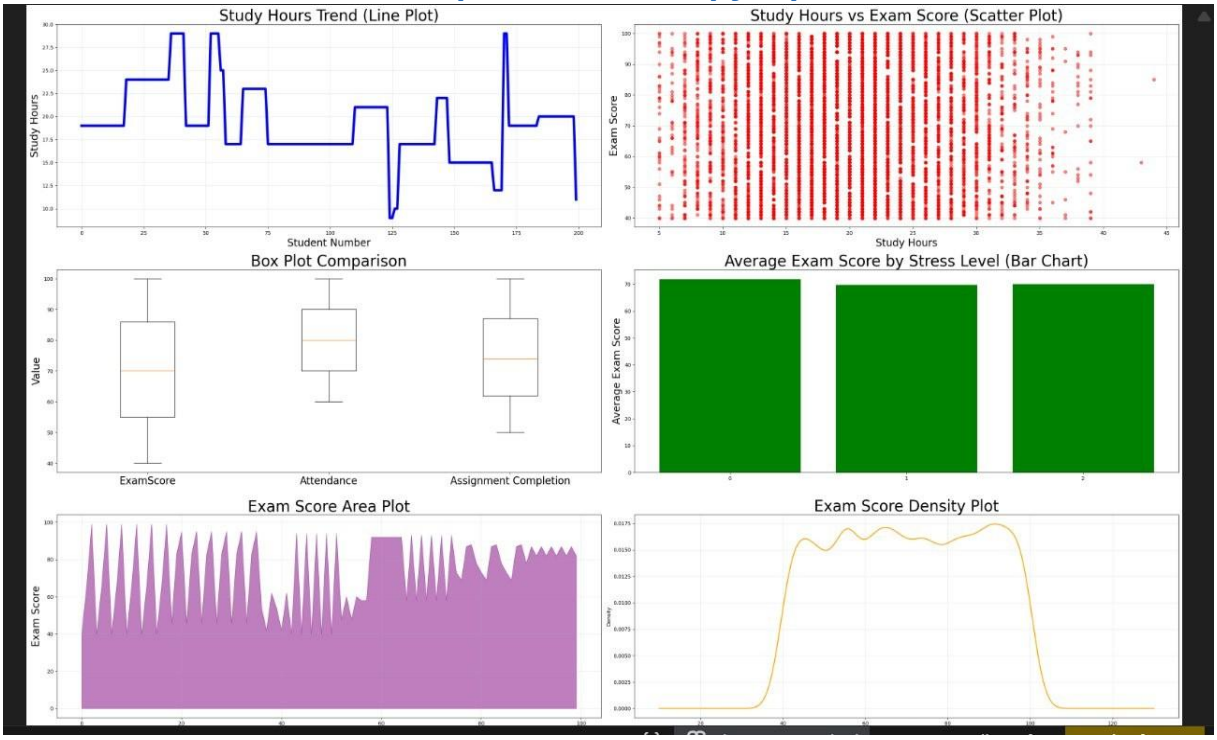


Figure B.3: Six EDA charts generated in the Jupyter Notebook — Study Hours Trend, Study Hours vs Exam Score (scatter), Box Plot comparison, Average Exam Score by Stress Level, Exam Score Area Plot, and Exam Score Density Plot.

B.4 — Confusion Matrix Output (Exam_results.ipynb)

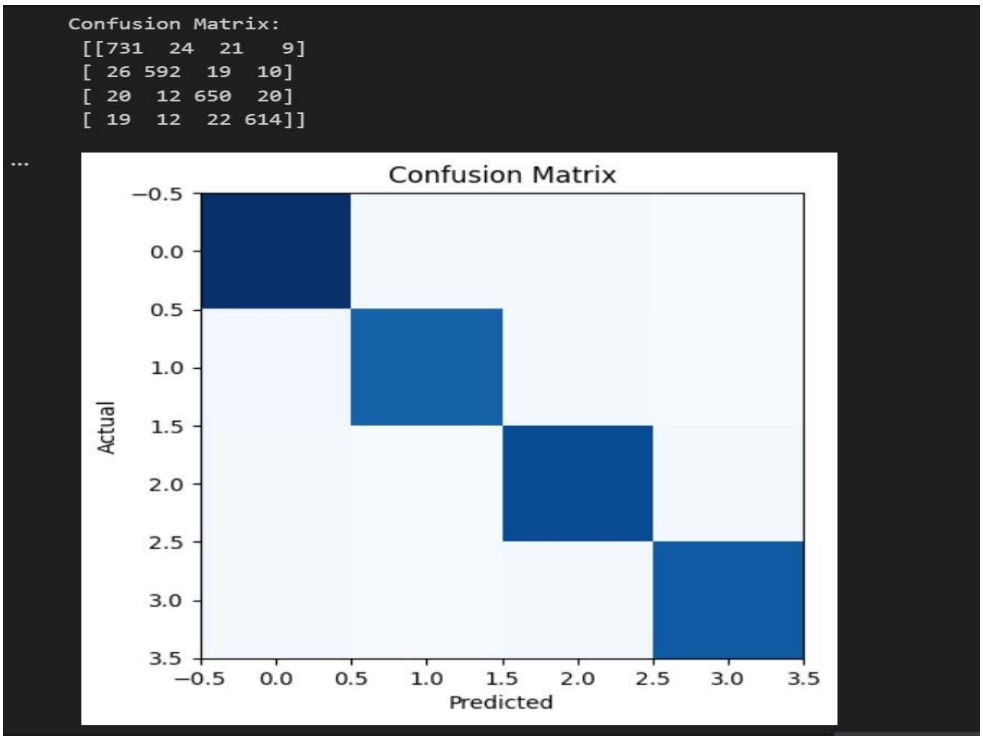


Figure B.4: Confusion matrix generated for the Random Forest model. Raw values shown above the plot: [[731,24,21,9],[26,592,19,10],[20,12,650,20],[19,12,22,614]]. Overall model accuracy: 92%.

Appendix C: Glossary

Accuracy: The proportion of all test records classified correctly by the model. This project's Random Forest achieved 92% accuracy.

Assignment Completion: The percentage of assignments a student submitted on time; one of the 9 input features.

best_random_forest_model.pkl: The serialized trained Random Forest model saved using Joblib, loaded by the Flask application.

Bootstrap Sampling: The process of creating random samples with replacement from the training data, used by Random Forest to build diverse trees.

Categorical Variable: A variable taking discrete values (e.g., Gender: Male/Female, Stress Level: Low/Medium/High).

Classification: A supervised ML task where the output is a discrete category — in this project: Excellent, Good, Satisfactory, or Fail.

Confusion Matrix: A 4×4 table comparing actual vs. predicted performance categories; rows are actual, columns are predicted.

Cross-Validation: Evaluating a model by splitting data into multiple train/test folds to produce more reliable performance estimates.

Decision Tree: A tree-shaped model making predictions via a series of feature-based decision rules; evaluated but not selected as the final model.

EDM (Educational Data Mining): The field of applying data mining and ML to educational data to improve learning outcomes.

Encoding: Converting categorical values to numbers for ML model input (e.g., Male=0, Female=1).

Ensemble Method: A technique combining multiple ML models; Random Forest is an ensemble of Decision Trees.

Exam_results.ipynb: The Jupyter Notebook used for EDA, model training, evaluation, and confusion matrix generation.

Excellent: Performance category for students scoring 85–100%; the highest grade band in this system.

F1-Score: Harmonic mean of precision and recall; relevant when class imbalance exists across grade categories.

Fail: Performance category for students scoring below 50%; the at-risk group this system helps identify early.

Feature: A measurable input attribute used by the model (e.g., Hours Studied, Attendance %).

Feature Importance: Scores from Random Forest indicating how much each feature contributes to prediction accuracy.

Flask: Lightweight Python web framework used to build the prediction web application (app.py).

Good: Performance category for students scoring 70–84%.

GridSearchCV: Scikit-learn method for exhaustive hyperparameter tuning with cross-validation.

Joblib: Python library used to serialize (save) and deserialize (load) the trained model as best_random_forest_model.pkl.

Logistic Regression: First model evaluated; achieved only 20% accuracy due to the non-linear nature of the data; discarded.

Matplotlib: Python library used for generating EDA charts including line plots, scatter plots, and box plots.

NumPy: Python library for efficient numerical array operations, used during pre-processing and prediction.

Ordinal Encoding: Encoding for ordered categories, preserving natural order (Low=0, Medium=1, High=2).

Overfitting: When a model memorizes training data and performs poorly on new data; reduced in Random Forest via ensemble.

Pandas: Python library for data manipulation; used to load, clean, and process student_performance.csv.

Random Forest: The selected model achieving 92% accuracy; builds many Decision Trees and votes on predictions.

Satisfactory: Performance category for students scoring 50–69%.

Scikit-learn: Python ML library providing all classifier implementations, evaluation metrics, and pre-processing tools.

Seaborn: Python statistical visualization library used for the confusion matrix heatmap and EDA plots.

Stratified Split: Train-test split that preserves the proportion of each class (Excellent/Good/Satisfactory/Fail) in both sets.

student_performance.csv: The dataset: 14,003 student records, 17 columns, used for training and testing the model.

Appendix D: Project Summary Card

Attribute	Details
Project Title	Exam Results Prediction System
Type	Long-Term Internship — Machine Learning Using Python
Institution	Aditya Degree College, Kakinada, Andhra Pradesh
University	Adikavi Nannaya University (NAAC A++ Grade)
Programme	Bachelor of Science in Data Science (BSc Data Science) — III BSc Data Science
Academic Year	2023–2026
Team	Samantha Ndlovu, Prosper Tasiyana, Desire Denge, Nkosana Brighton Chirokoto
Guide	Ms.R.Naga Venkata Harika — Vidurutha Solutions (India & Canada)
Dataset	student_performance.csv — 14,003 rows, 17 columns
Input Features	9 features: study hours, attendance, resources, internet, gender, age, assignment completion, stress, online courses
Output	4 categories: Excellent (85-100%), Good (70-84%), Satisfactory (50-69%), Fail (<50%)
Models Evaluated	Logistic Regression (20%), Decision Tree (~82%), Random Forest (92%)
Final Model	Random Forest — 92% accuracy, saved as best_random_forest_model.pkl
Confusion Matrix	[[731,24,21,9],[26,592,19,10],[20,12,650,20],[19,12,22,614]]
Web Application	Flask — runs locally at http://127.0.0.1:5000 via VS Code

Key Files	app.py, best_random_forest_model.pkl, Exam_results.ipynb, student_performance.csv
Repository	github.com/benjiie92/exam-results
Deployment Status	Fully working locally — not yet deployed publicly
Top Finding	Study hours, attendance, and assignment completion are strongest grade predictors
Next Goal	Cloud deployment and LMS integration